Essentials

MCP servers

Connect your agent to your own tools and prompts using the Model Context Protocol (MCP).

What is MCP?

The **Model Context Protocol (MCP)** is an open standard for exposing **tools** (callable actions) and **prompts** (reusable instructions) over a well-defined protocol. Any MCP client can connect to a server, discover what it offers, and use it.

In Leaping, you can point an agent at **your own** MCP server(s) so it can call your tools and use your prompts — **without us hand-building a custom integration for you**. If your team already runs (or can stand up) an MCP server, you self-serve the whole setup from Studio:

- Register one or more MCP servers on an agent (URL + auth).
- **Discover** the server's tools and prompts.
- **Test** a tool and **preview** a prompt before publishing.
- Expose selected tools to specific conversation stages.
- Source a system or stage message from an MCP prompt.

> MCP tools behave just like native functions inside a conversation. Once a tool is exposed on a stage, the agent decides when to call it, executes it, and uses the result — exactly as it does with built-in functions.

Connecting a server

Open your agent and go to the **MCP Servers** tab (currently in **beta**). Click **Add server** and fill in:

Field	Required	Purpose
Server Name	✅	How the server appears in Studio.
Server URL	✅	The remote **Streamable HTTP** endpoint (e.g. `https://mcp.example.com/mcp`).
Tool Prefix	—	Optional. Prefixes every tool name from this server (e.g. `crm` → `crm_lookup_order`) so tools never collide with native functions or tools from other servers.
Authentication	—	Optional. Toggle on to send credentials with every request.

Authentication

Two auth types are supported:

- **Bearer Token** — sent as an `Authorization: Bearer <token>` header.
- **Custom Header** — a header name (e.g. `X-API-Key`) and its value.

> Secrets (the bearer token / header value) are **encrypted at rest** — they are stored as ciphertext, never in plaintext, and are never returned to the browser after saving. Whenever you enter a new value and save, it is re-encrypted, so rotating a secret always takes effect.

Discovering tools and prompts

Once the URL (and auth, if any) is set, click **Discover**. Leaping connects to the server and lists everything it exposes under two tabs:

- **Tools** — each shows its (prefixed) name, description, and input parameters. Required parameters are marked with `*`.
- **Prompts** — each shows its name, description, and any arguments it accepts.

Discovery results are **cached on the agent** when you save, so the builder and validation can work without re-contacting your server.

> v1 supports **tools and prompts only** — MCP **resources** are not yet supported. The agent always uses the tools your server reports **live** at call time, so tools you add after publishing become available without re-publishing. The cached discovery is what powers the builder UI and publish-time validation.

Testing a tool

In the **Tools** tab, click the play (▶) icon on any tool to open a small test panel, fill in its arguments, and **Run Test**. The raw result (or error) is shown inline — the same content the agent would receive during a real conversation.

Previewing a prompt

In the **Prompts** tab, click the play (▶) icon on any prompt, fill in its arguments, and **Render Preview** to see exactly what the server returns.

Exposing tools to a stage

MCP tools are opt-in per stage, just like native functions. Open a node's editor and find the **MCP Tools** section (available on **Dialogue**, **Response**, and **Function** nodes):

- **Dialogue / Response nodes** — click the tool tags to toggle which tools the agent may call in that stage. You can select tools from multiple servers.
- **Function nodes** — select exactly **one** MCP tool to run deterministically. A function node is either a native function or a single MCP tool, not both. You can bind the tool's arguments to fields or literal values; anything left unbound is filled in by the agent at call time.

> Under the hood, each stage stores **which server and which tools** it may use. A stage can also be configured to expose **all** of a server's tools (including ones added after publishing) rather than a fixed list — useful when you want a stage to automatically pick up new tools as your server grows.

Using an MCP prompt as a message

When at least one connected server exposes prompts, a **Use MCP prompt** option appears wherever you author a message:

- The **system message** (Agent Setup).
- A **stage message** on Dialogue / Response nodes.
- The **script** on Scripted nodes (the resolved prompt text is emitted as the script).

Pick a server and a prompt, bind its arguments (to fields or literals), then click **Preview & save fallback**.

> On a **Scripted** node the MCP prompt is resolved to text and emitted as the node's message, using the same fallback chain below. With no MCP prompt set, the node plays its fixed script unchanged.

What-you-preview-is-what-you-get

When the agent runs, an MCP-sourced message is rendered **live** by your server using the real argument values. But servers can be slow or unreachable, so Leaping also keeps a **fallback**:

1. The text you see in the preview is captured and stored as the fallback (byte-for-byte).
2. Any argument bound to a field is sent during preview as a `{{field}}` **marker** instead of a real value, so the dynamic slot survives into the fallback.
3. At call time, those `{{field}}` markers are filled in with the live field values.

So the previewed text **is** the fallback: you review and approve the exact string that will be used if your server is unavailable, with `{{field}}` shown in place of values that get filled in at runtime.

> **Fallback order at runtime:** live render from your server → the saved preview (with `{{field}}` markers filled in) → the message you typed by hand. Because of this chain, you must still author a non-empty plain message — it's the last-resort fallback, and the agent will refuse to publish without it.

> If a preview fails (for example, your server validates or transforms the marker argument and rejects it), nothing is cached and the fallback degrades to your hand-authored message. The preview surfaces this immediately, so there are no surprises at call time.

Marker arguments need passthrough interpolation

A field-bound prompt argument is sent to your server as the literal string `{{field_name}}` (the marker), both at preview time and when an MCP prompt drives the **system message** at the start of a call. This only works if your server **interpolates the argument value verbatim** into the prompt text.

If your server instead **validates, coerces, or branches on** that argument (for example it expects an enum, a number, or a date, or its templating errors on an unknown token), it may reject or mangle the `{{field_name}}` marker. When that happens the render fails, nothing is cached, and the message falls back to your hand-authored static text — safe, but **without** the dynamic field value.

Guidance: bind an argument to a field only when the server passes that argument straight through into the text. Otherwise use a literal value, or rely on the static fallback.

System message specifics

The **system message** is resolved **once at the start of the call**, before the conversation begins. Because of that timing:

- **Field arguments are re-evaluated every turn.** The system message keeps the `{{field}}` markers and fills them with the current field values on each turn, so a field updated early in the call (for example by a pre-conversation step) is reflected automatically — your server is only contacted once.
- **LLM-extracted arguments are not allowed** for the system message. There's no conversation to extract from yet, so the **LLM** argument option is hidden for the system prompt — bind its arguments to **fields or literals** only. (Stage messages, which resolve once you're in the stage, do support LLM-extracted arguments.)

> Changes to the system MCP prompt in the Agent Setup editor follow the same **Save / Cancel** rules as the rest of that dialog: they only take effect when you **Save**, and **Cancel** discards them.

Security model

MCP servers are external, customer-supplied endpoints, so Leaping applies several protections to every connection — both from the builder and at call time:

- **Encrypted secrets** — bearer tokens and header values are stored as ciphertext, never plaintext, and never sent back to the browser.
- **SSRF egress guard** — outbound connections are blocked if the URL resolves to a private, reserved, loopback, link-local, or cloud-metadata address (e.g. `127.0.0.1`, `10.x`, `192.168.x`, `169.254.169.254`). This is enforced, and also closes DNS-rebinding and redirect-to-internal tricks.
- **Timeouts** — connect, read, tool-call, and prompt-render operations each have a strict time budget, so a slow or hung server can't stall a live conversation.
- **Response-size cap** — tool and prompt responses are capped (256 KiB) so a server can't exhaust memory or flood the model's context.
- **Remote-only** — only remote **Streamable HTTP** servers are supported. Local/stdio servers are not allowed.
- **Multi-tenancy** — MCP configuration lives on your agent and inherits its existing access controls; it is never shared across organizations.

Limitations (v1)

- **Tools and prompts only** — MCP **resources** are not yet supported.
- **Static auth only** — bearer token or custom header. OAuth flows are not supported.
- **Remote servers only** — no local/stdio transport.
- Servers are part of the **published agent config** — they're registered in Studio, not added dynamically per call.

Ask a question...
⌘ /
Powered by mintlify